



Robotaksi Hibrit Behavior Tree v3 — Sıfırdan İleri Seviye Eğitim Rehberi

Bu rehber, mimariyi sıfırdan anlamak, neyin nerede çalıştığını çözmek ve "C++ tarafında ben ne yazacağım?" sorusuna kesin yanıtlar bulmak isteyenler için yazıldı. Ayrıca şartnamedeki keskin ceza kurallarının (hız limitleri, yanlış park vb.) sisteme nasıl işlendiğini detaylandırır.

1. BÜYÜK RESİM: Bu Mimari Hem Turları Hem "Genel Sürüşü" Nasıl Çözer?

Bazen akla şu soru gelir: "Biz yarışmada birebir 'Tur 1, Tur 2' sırasıyla gitmeyeceğiz, bazen sadece genel sürüş yapacağız. Bu sistem sadece turlar için mi çalışıyor?"

Cevap: Hayır. Sistem tamamen dinamik bir "Liste (Segment) Tüketici"dir. Robotun ne yapacağını belirleyen şey, BT'nin kendisi değil, en başta `LoadMission` düğümünün okuduğu `geojson_file` (harita/görev dosyası) dır.

- Eğer harita dosyası sadece `[LANE_FOLLOW, LANE_FOLLOW, INTERSECTION, LANE_FOLLOW]` veriyorsa, robot sadece genel sürüş yapar.
- Eğer harita dosyası `[... PASSENGER_STOP, PARKING]` gibi duraklama noktaları veriyorsa, robot o noktalarda yolcu alır veya park eder. Yani bu Behavior Tree, senaryoya değil **yolun parçalarına** göre tepki verir. Haritaya ne yazarsan onu oynatır.

Mimarinin 3 Temel Yasası (v3 Güncellemeleriyle):

- Güvenlik HER YERDEDİR:** Acil stop, yaya geçidi, dinamik engeller, trafik ışığı, DUR tabelası, şerit birleşmesi gibi kurallar belirli bir kavşakta değil, **yolun her santimetresinde** kontrol edilir (`SafetyReflexes`).
- Latch (Kilit) Mekanizması:** Robot bir DUR tabelasında durup kalktığında, tabela hâlâ kamerasındaysa tekrar tekrar durup kalmasın diye "bunu hallettim" bayrakları (`ClearHandledFlags`) ile segmentler arası geçiş kontrol altında tutulur.
- Yürüyüş Hızı Zorunluluğu:** Şartname gereği araç yürüyüş hızından (~ 1.4 m/s) hızlı olmak zorundadır. Bu yüzden `cruise_speed = 1.50` m/s olarak sabitlenmış, sadece manevra ve kavşaklarda $0.30 - 0.50$ değerlerine düşülmüştür.

2. AĞAÇLARIN (SUBTREE) TEK TEK İNCELENMESİ

Burada sistemde yer alan 17 ağacı 4 ana gruba ayırıp inceleyeceğiz.

GRUP A: Temel Kontrol ve Görev Başlatıcılar

1. MainTree (Ana Ağaç)

- Mantığı:** Robot çalıştığında ilk burası okunur.
- İçindeki Adımlar:**
 - `SetBlackboard` : Sistem hızlarını ayarlar (`cruise_speed = 1.50` , `parking_speed = 0.30` m/s).
 - `LoadMission` : Haritadan segment rotasını çıkarır.
 - `WaitForGoSignal` : Şartname gereği aracın harekete geçmesi için dışarıdan verilecek "UMS-2 Start/Go" sinyalini bekler.

- `DriveWithRecovery` (Fallback) : Rotayı sürmeyi dener, kilitlenirse `Recovery` çalıştırır.

2. DriveAllSegments & 3. ExecuteSegment

- **Mantığı:** Rotadaki segmentleri sırayla bitene kadar döndüren "for-loop" (`DriveAllSegments`) ve her segmentin adını okuyup ilgili ağaca yönlendiren Switch-Case (`ExecuteSegment`) yapılarıdır. Geçilen her segmentin başında `ClearHandledFlags` çalışarak eski tabela kayıtlarını (örneğin eski bir trafik ışığı bilgisini) hafızadan siler.

4. Recovery

- **Mantığı:** Robot fiziksel olarak sıkışmışsa (`IsStuck` algılandığında), geriye doğru 30 cm gitmesini (`BackUp`) ve olduğu yerde dönmesini (`Spin`) söyler. Kendini kurtarmaya çalışır.

GRUP B: Hareket ve Güvenlik Çekirdeği (En Önemli Kısım!)

5. SafeDrive

- **Mantığı:** Hareketi başlatan çekirdektir. `ReactiveSequence` olarak çalışır.
- **İşleyiş:** Saniyede defalarca önce `SafetyReflexes` kalkanına bakar. Tehlike yoksa, +30 puan veren `RecordMissionPoint` (görev noktası kaydı) kontrolünü yapar ve `FollowLaneSegment` ile direksiyonu çevirir. Yolda tehlike çıkarsa sürüş anında kesilir.

6. SafetyReflexes (Güvenlik Kalkanı)

- **Mantığı:** 7 katmanlı savunma hattı.
- **Katmanlar:**
 1. `EmergencyStopRequested` : Acil durum butonu basılı mı?
 2. `PedestrianAhead` : Fiziksel yaya var mı? Dur ve bekle.
 3. `DynamicObstacleAhead` : Dinamik araç/bisiklet var mı? Dur ve bekle.
 4. `StaticObstacleInLane` : Statik engel var mı? `HandleStaticObstacle` ağacına git.
(Aşağıdaki katmanlar sadece $Tur \geq 2$ ise aktiftir, $Tur 1$ 'de ortamda levha olmadığı için fantom duruşları engeller):
 5. `TrafficLightAhead` : Trafik ışığına göre dur/kalk (`HandleTrafficLight`).
 6. `StopSignAhead` : DUR tabelasında 2sn dur ve devam et.
 7. `GlobalRoadSignAhead` : Şerit kapanması (B-50h/i), yaya geçidi levhası (B-14a), girilmez levhası (TT-4) gibi durumları yönetir (`HandleRoadSigns`).

GRUP C: Tehlike Aşım Alt-Ağaçları

7. HandleStaticObstacle

- **Mantığı:** Sabit bir engelle karşılaşınca ne yapılacağını seçer.
- **İşleyiş:**
 - Önce yolun tek yön mü çift yön mü olduğuna bakar (`IsTwoWayRoad`). İki yönlü yolsa, karşıdan gelen trafik riskinden dolayı şerit değiştirmez!
 - Yan şerit boşsa (`AvoidanceSpaceAvailable`), `CalculateLaneChange` ile direksiyonu yana kırar.
 - Yan şerit doluysa VEYA yol iki yönlüyse **şartname gereği** o engeli geçmeye zorlamaz. `ReplanRoute` ile kendisine engelsiz yeni bir alternatif rota çizer.

8. HandleTrafficLight

- **Mantığı:** Kırmızıda dur, yeşilde 5sn içinde geç kuralını (şartname: +100 puan) uygular.

- **İşleyiş:** Işık kırmızıysa `StopAtStopLine` ile çizgiye 5m mesafede durur. Yeşil yandığında `ProceedOnGreen` ile 5 saniye toleransı içinde hızla kalkış yapar.

9. HandleRoadSigns

- **Mantığı:** Global tabelalara tepki verir.
- **İşleyiş:**
 - B-50h/i (Şerit kapanıyor): `HandleStaticObstacle` yapısını tetikler.
 - B-14a (Yaya Geçidi Tabelası): Sadece yaya varsa hızı 0.30'a düşürür ve bekler. Yaya yoksa gereksiz yavaşlama yapmaz.
 - TT-36a/b (Sağdan/Soldan Gidiniz) & TT-4 (Girilmez): Hatalı yola girilmemesi için `ReplanRoute` ile rotayı günceller.

GRUP D: Segment Davranışları (O Yere Özel Kurallar)

10. Seg_LaneFollow

- Dümdüz yolda gidiş. `cruise_speed` (1.50) hızında `SafeDrive` 'ı çağırarak ilerler.

11. Seg_Intersection & 12. ValidateTurn

- **Mantığı:** Kavşağa girerken yavaşlar (0.50).
- **İşleyiş:** Kavşağa girmeden hemen önce plandaki dönüş yönüyle tabelaların çelişip çelişmediğine (`TurnConflictsWithSigns`) bakar. Örn: "Sola dönüş planı" ama "TT-26 Sola Dönülmez" tabelası varsa, inat etmez, `ReplanRoute` çağırarak güzergahı değiştirir.

13. Seg_Roundabout (Dönel Kavşak)

- Dönel kavşak girişinde hızı düşürür ve `YieldAtRoundabout` ile içeride dönen araçları algılayıp boşluk bekler. Boşluk varsa kavşağa girer.

14. Seg_Tunnel (Tünel)

- Şartnamedeki +200 puanlık görev. Tünele girmeden farları açar (`TurnHeadlights on`), çıkışta kapatır (`TurnHeadlights off`).

15. Seg_PassengerStop (Yolcu Alma Durağı)

- **Mantığı:** Konum ve kafa açısı hassasiyeti (+70 puan).
- **İşleyiş:** Hedefe yaklaşır, durur. `CheckStopAccuracy` ile hedefe mesafe (1m) ve kafa açısı (`heading_tolerance=15.0` derece) limitleri içinde miyiz diye bakar. Yamuk durmuşsak ufak manevralarla pozisyon düzeltir. Sonra 15-20 saniye bekler ve PLC sinyalinin gönderir.

16. Seg_Parking (Dik Park)

- **Mantığı:** Park alanı seçimi ve park manevrası (+100 puan).
- **İşleyiş:** Park alanına girişi kaydeder. `FindParkingSlot` ile kameradan P-3a tabelası olan, P-1 (Park Yasağı) tabelası olmayan ve içi fiziksel olarak boş olan slotu bulur. Sonra `ExecuteParking` ile 3 dakikalık limit içinde geri park manevrasını yapar.

3. İMPLEMENTASYON REHBERİ (C++ Tarafı)

XML mimarisi yarışma kurallarına %100 uyan, bug'lardan arındırılmış bir "Beyin" haline getirildi. Senin görevin `segment_bt_nodes.cpp` içindeki `// TODO` alanlarını gerçek ROS 2 topic'leriyle (sensör/motor) konuşturmak.

Nelere Dikkat Edilecek?

1. "CalculateLaneChange" ve Engelden Kaçma: Engelden kaçarken veya şerit birleşmesinde, geçeceğin hedef şeritteki çizgi tipini (kesikli mi, düz mü?) veya o anki tabelaları (örn: sağdan gidiniz) parametre olarak al (`detected_signs`). Düz çizgi varsa şerit değiştirme, `FAILURE` dön. Sistem zaten alternatif rota (`ReplanRoute`) üretecektir.

2. FollowLaneSegment (Nav2 mi, Vision mı?) Bu fonksiyon sürüşün kalbidir. Kameradan şerit takibi (PID ile direksiyon açısı hesabı) yapıyorsan, bu node içinde `cmd_vel` mesajı basılmalıdır. Asla içeride `while(true)` gibi blocking döngüler kurma. StatefulActionNode mimarisi kullanılarak `onRunning()` fonksiyonu içinde tick bazlı dönüşler yapılmalıdır.

3. "ClearHandledFlags" (Latch Mantığı) Aynı DUR tabelasının dibinde dur-kalk-dur-kalk sonsuz döngüsüne girmemek için bu node hayati önemdedir. Kamera bir DUR tabelasını algıladığında C++ tarafında bir state/bayrak tut. O tabela için bir kere durulduysa, o bayrağı "işlendi" olarak işaretle. `ClearHandledFlags` node'u tetiklendiğinde (yeni segmente geçildiğinde) bu bayrakları tekrar `false` konumuna getir.

4. FindParkingSlot Şartları Kamera ve Lidar verilerini birleştirerek (Sensor Fusion) şu şartları sağlayan slotları bul:

- Kamerada o park yeri için P-3a (Park Yeri) Bounding Box'ı olmalı.
- P-1 (Park Yapılmaz) Bounding Box'ı OLMAMALI.
- Lidar/Kamera derinlik haritasında slotun içi boş (engel yok) olmalı.

Sistem, harita/turlar arası uyumsuzlukları ve puan kırma tuzaklarını savuşturacak şekilde tasarlandı. Bundan sonrası tamamen temiz bir C++ ROS 2 entegrasyonundan ibaret. Başarılar!